

Capitolo 2

Introduzione alle query in MongoDB

Nel capitolo precedente abbiamo costruito il contesto: abbiamo visto cos'è MongoDB, dove si colloca nel panorama dei database e abbiamo preparato il database di esempio che ci accompagnerà per tutto il libro. Adesso cominciamo davvero a mettere le mani sulle query.

Questo capitolo ha un obiettivo molto preciso: chiarire che cosa significa fare una query in MongoDB e distinguere bene tre aree che i principianti spesso mescolano tra loro: le operazioni CRUD, il linguaggio di interrogazione e il framework di aggregazione.

Capire questa distinzione all'inizio ti farà risparmiare molta confusione più avanti.

2.1 Cos'è una query in MongoDB

Nel linguaggio comune, una query è una richiesta fatta al database. È una definizione semplice, ma corretta.

Quando diciamo fare una query potremmo intendere cose diverse: leggere alcuni documenti, inserire nuovi documenti, aggiornare documenti esistenti, cancellare documenti oppure trasformare e aggregare dati per ottenere report o analisi.

MongoDB raggruppa le operazioni fondamentali nella famiglia CRUD, cioè Create, Read, Update e Delete. Questo è già un primo punto importante: la stessa idea di selezionare i documenti giusti ritorna più volte. Se sai filtrare bene i documenti in lettura, hai già una parte del bagaglio mentale che ti servirà anche per aggiornare o cancellare.

2.2 CRUD, in pratica: i quattro verbi fondamentali

CRUD è un acronimo molto usato nel mondo dei database e delle applicazioni: Create per creare nuovi documenti, Read per leggerli, Update per modificarli e Delete per eliminarli.

Per rendere l'idea, pensa a una collection chiamata clients. Quando inserisci un nuovo cliente, stai facendo Create. Quando cerchi tutti i clienti della provincia di Verona, stai facendo Read. Quando cambi lo stato di un cliente da PROSPECT ad ACTIVE, stai facendo Update. Quando elimini clienti di test creati per sbaglio, stai facendo Delete.

Detta così sembra banale, ma è importante fissare questo schema mentale, perché tutto il libro ruoterà continuamente attorno a questi quattro verbi.

2.3 Che cosa sono database, collection e documenti

Prima di continuare, fermiamoci un attimo su tre concetti di base che useremo in ogni pagina del libro.

MongoDB memorizza i dati come documenti BSON, raggruppati in collection, a loro volta contenute in un database. Se arrivi dal mondo SQL, una buona mappa mentale iniziale è questa: tabella uguale collection, riga uguale documento, colonna uguale campo. Non è una traduzione perfetta, ma come primo orientamento funziona bene.

Un documento MongoDB può contenere campi semplici, documenti annidati, array e tipi specifici come date e ObjectId. Questo spiega perché MongoDB risulti molto comodo quando i dati hanno una forma gerarchica o semi-strutturata.

2.4 La sintassi generale delle operazioni in MongoDB

MongoDB, nella shell mongosh, usa una sintassi molto leggibile. Di solito si parte dalla collection e si richiama un metodo.

La forma generale è questa:

```
db.nomeCollection.metodo(...)
```

Per esempio:

```
db.clients.find(...)
db.products.insertOne(...)
db.salesOrders.updateMany(...)
db.suppliers.deleteOne(...)
```

Questa coerenza sintattica è molto utile perché ti abitua a un pattern costante: cambiando il metodo, cambi il tipo di operazione; cambiando la collection, cambi il contesto dei dati.

Un altro dettaglio utile è che `find()` restituisce un cursor, non direttamente un array già materializzato. Più avanti vedremo bene che cosa implica, ma già adesso è utile sapere che dopo `find()` si possono concatenare operazioni come `sort()` o `limit()`.

2.5 Filtro, proiezione, ordinamento e pipeline

Una delle fonti di confusione più comuni per chi inizia è mescolare concetti che in realtà hanno ruoli diversi. Vale la pena separarli con calma.

Il filtro serve a dire quali documenti vuoi selezionare. La proiezione serve a dire quali campi vuoi vedere del documento. L'ordinamento risponde alla domanda in che ordine vuoi vedere i documenti. La pipeline, invece, è un concetto più avanzato: non si limita a scegliere documenti o campi, ma li trasforma in una sequenza di passaggi.

Se fai una `find()`, stai dicendo qualcosa come: fammi vedere gli ordini confermati del 2025. Se fai una `aggregate()`, potresti dire: prendi gli ordini confermati del 2025, raggruppalì per mese, somma il fatturato e restituisci il totale mensile. Nel primo caso stai leggendo documenti. Nel secondo stai costruendo un processo di trasformazione dei dati.

2.6 MongoDB Query Language: che cosa significa davvero

Quando si parla di MongoDB Query Language, o MQL, si intende l'insieme di comandi, operatori e costrutti con cui interroghi e manipoli i dati in MongoDB.

Detto in modo semplice: MQL non è solo il filtro della `find()`. È il linguaggio operativo con cui descrivi quali documenti selezionare, come aggiornarli, come cancellarli, come aggregarli e come comporre trasformazioni più complesse.

In altre parole, MQL è il vocabolario; CRUD e Aggregation sono due grandi contesti in cui quel vocabolario viene usato.

2.7 find() e aggregate(): la distinzione che devi capire subito

Se dovessi scegliere una sola idea da fissare bene in questo capitolo, sceglierei questa: find() e aggregate() non sono intercambiabili, anche se a volte sembrano vicini.

Usa find() quando vuoi leggere documenti. Usa aggregate() quando vuoi trasformare, riassumere, raggruppare o analizzare.

Per esempio, se vuoi vedere tutti i prodotti attivi della categoria ACCESSORIES, find() è perfetta. Se vuoi sapere il numero di prodotti attivi per categoria, ordinati dal più numeroso al meno numeroso, entra in gioco aggregate().

2.8 Le query non riguardano solo la lettura

Nel linguaggio comune, quando uno dice scriviamo una query, spesso pensa soltanto alla lettura dei dati. In MongoDB, però, questa visione è troppo stretta. Lavorare con il database significa anche creare nuovi documenti, aggiornare quelli esistenti e cancellare quelli che non servono più.

La cosa interessante è che queste operazioni non sono mondi separati. Spesso condividono la stessa logica di base: prima individui i documenti giusti, poi decidi cosa farci. Nella lettura li visualizzi, nell'update li modifichi, nella delete li elimini. Nel create, invece, costruisci direttamente un nuovo documento e lo aggiungi alla collection.

Facciamo subito qualche esempio molto semplice usando il database aziendale di esempio.

Esempio di lettura con find()

Supponiamo di voler vedere alcuni clienti attivi:

```
db.clients.find({ status: "ACTIVE" }).limit(5)
```

Questa query chiede a MongoDB di cercare nella collection clients tutti i documenti che hanno status uguale a "ACTIVE" e di mostrarne al massimo cinque.

Esempio di creazione con insertOne()

Adesso inseriamo un nuovo cliente di prova:

```

db.clients.insertOne({
  code: "CLI9999",
  companyName: "Cliente Demo SRL",
  type: "BUSINESS",
  status: "PROSPECT",
  email: "info@clientedemo.it",
  phone: "+39-045-000000",
  category: "BRONZE",
  creditLimit: 5000,
  paymentTermsDays: 30,
  addresses: [
    {
      type: "LEGAL",
      street: "Via Esempio 10",
      city: "Verona",
      province: "VR",
      postalCode: "37121",
      country: "IT"
    }
  ],
  contacts: [
    {
      name: "Mario Rossi",
      role: "Buyer",
      email: "mario.rossi@clientedemo.it",
      phone: "+39-045-000111"
    }
  ],
  tags: ["demo", "new"],
  createdAt: new Date(),
  updatedAt: new Date()
})

```

Dopo l'inserimento, possiamo verificare che il cliente esista davvero:

```
db.clients.find({ code: "CLI9999" })
```

Esempio di aggiornamento con updateOne()

Ora immaginiamo di voler trasformare questo cliente da semplice prospect a cliente attivo:

```

db.clients.updateOne(
  { code: "CLI9999" },
  {
    $set: {
      status: "ACTIVE",
      category: "SILVER",
      updatedAt: new Date()
    }
  }
)

```

Per vedere il risultato:

```
db.clients.find({ code: "CLI9999" })
```

Esempio di cancellazione con deleteOne()

Se il cliente di prova non ci serve più, possiamo eliminarlo:

```
db.clients.deleteOne({ code: "CLI9999" })
```

Un piccolo flusso prudente, soprattutto all’inizio, può essere questo:

```
db.clients.find({ code: "CLI9999" })
db.clients.deleteOne({ code: "CLI9999" })
db.clients.find({ code: "CLI9999" })
```

2.9 Un primo assaggio di aggregate()

Accanto alle operazioni CRUD esiste poi un’altra grande famiglia di elaborazioni: le aggregazioni. Per il momento non dobbiamo ancora entrare nei dettagli. Ci basta vedere un esempio molto semplice, giusto per prendere confidenza con l’idea.

Supponiamo di voler sapere quanti clienti ci sono per stato. Non ci interessa leggere un cliente alla volta. Ci interessa ottenere un piccolo riepilogo, per esempio quanti sono ACTIVE, quanti sono PROSPECT e quanti sono SUSPENDED.

Una pipeline di aggregazione molto semplice potrebbe essere questa:

```
db.clients.aggregate([
  {
    $group: {
      _id: "$status",
      totalClients: { $sum: 1 }
    }
  }
])
```

Per adesso leggila così: prendi i documenti della collection clients, raggruppalì in base al valore del campo status e per ogni gruppo conta quanti documenti ci sono.

Questo esempio è utile perché mostra subito la differenza rispetto a find(). Con find() chiedi: fammi vedere dei documenti. Con aggregate() chiedi: prendi i documenti e costruisci da essi un risultato riassuntivo.

Non preoccuparti se la sintassi non ti è ancora del tutto familiare. Tutto verrà spiegato con molta più calma nei capitoli successivi, quando entreremo davvero nel mondo delle pipeline di aggregazione.

2.10 Perché questi primi esempi sono importanti

Questi esempi iniziali sono volutamente piccoli. Non servono ancora a mostrare tutta la potenza di MongoDB. Servono a creare familiarità.

Il lettore, già a questo punto, può leggere alcuni clienti con find(), inserire un nuovo cliente di prova con insertOne(), modificarlo con updateOne(), cancellarlo con deleteOne() e vedere un primo riepilogo con aggregate().

È un passaggio importante, perché trasforma il libro da spiegazione teorica a esperienza pratica.

2.11 Update classico e update con pipeline

MongoDB permette di aggiornare documenti in almeno due modi concettualmente diversi: con i classici operatori di update, come \$set, oppure con una vera e propria aggregation pipeline di update.

Con un update classico dici qualcosa come: imposta questo valore. Con un update via pipeline puoi dire qualcosa come: calcola un nuovo valore in base a campi già presenti nel documento.

Per adesso ti basta sapere che MongoDB non si ferma alla logica trova e imposta: può anche eseguire trasformazioni più intelligenti durante l'aggiornamento.

2.12 Aggregation Framework: il motore delle query complesse

L'aggregation framework è uno dei punti di forza più importanti di MongoDB.

Detta in modo semplice, serve quando vuoi fare cose come contare, sommare, raggruppare, classificare, rimodellare i documenti, costruire report e analizzare dati temporali.

Per esempio, con una query semplice puoi leggere tutti gli ordini di febbraio. Con una pipeline di aggregazione puoi prendere gli ordini di febbraio, scomporre le righe prodotto, raggrupparle per categoria merceologica, sommare il fatturato e ordinare il risultato in ordine decrescente. Questo non è più un semplice leggere documenti. È analisi.

2.13 Il ruolo di \$group

Tra gli stage di aggregazione, \$group è uno dei più importanti.

In pratica, \$group è il meccanismo con cui rispondi a domande come: quanti clienti ci sono per categoria, quanto abbiamo fatturato per mese, quanti ordini ha gestito ogni venditore oppure qual è il totale delle quantità vendute per prodotto.

Se vieni da SQL, puoi pensare a \$group come al parente concettuale di GROUP BY. Non è identico in tutto, ma il ragionamento di base è molto simile.

2.14 Query semplici e query complesse

Una delle trappole più comuni è voler usare sempre la soluzione più potente. È un errore.

Una find() con filtro e proiezione è spesso la scelta giusta per consultazioni operative semplici. Una pipeline aggregate() diventa la scelta giusta quando hai bisogno di trasformare davvero il dataset.

Per esempio: mostrami i fornitori italiani attivi è una query semplice. Dimmi il numero di fornitori attivi per paese è già un'aggregazione.

2.15 Perché MongoDB usa BSON e non JSON puro

Spesso si dice che MongoDB salva JSON. In realtà è una scorciatoia linguistica. MongoDB memorizza i dati come documenti BSON, non come JSON testuale puro.

Questo è utile da sapere perché spiega alcuni comportamenti che altrimenti sembrerebbero strani: un identificatore _id spesso appare come ObjectId, le date non sono semplici stringhe e alcuni confronti dipendono anche dal tipo, non solo dal valore visivo.

In altre parole, il fatto che MongoDB lavori in BSON rende il database più espressivo e più adatto a gestire dati reali, ma richiede anche attenzione ai tipi.

2.16 Un primo sguardo operativo sul database aziendale

Anche se i prossimi capitoli entreranno davvero nel dettaglio della sintassi, possiamo già fare qualche ragionamento sul nostro database di esempio.

Abbiamo collection come clients, suppliers, products, salesOrders, purchaseOrders, warehouses, inventoryMovements ed employees. Su ciascuna di queste collection potremo fare tutti e quattro i tipi di operazioni CRUD. E su molte di esse potremo anche costruire aggregazioni significative.

Per esempio, su clients faremo query per stato, categoria, città, tag e limiti di credito; su products faremo ricerche per categoria, brand, prezzo, fornitore e soglia di riordino; su salesOrders faremo aggregazioni per mese, cliente, venditore, canale e prodotto.

2.17 Un principio fondamentale: prima capire il dato, poi scrivere la query

Questo principio vale in MongoDB, ma in realtà vale in qualsiasi database.

Molti errori non nascono da un problema di sintassi. Nascono dal fatto che chi scrive la query non ha capito abbastanza bene la forma dei dati.

Ogni volta che affronti una nuova collection, fermati e chiediti: quali sono i campi principali, ci sono documenti annidati, ci sono array, ci sono riferimenti ad altre collection, sto cercando documenti o sto cercando un risultato derivato? Queste domande ti faranno scrivere query migliori già da subito.

2.18 Conclusione del capitolo

In questo capitolo abbiamo costruito il vocabolario di base che ci servirà da qui in avanti.

Abbiamo visto che una query, in MongoDB, non è solo una lettura: rientra nel più ampio quadro delle operazioni CRUD, che comprendono creazione, lettura, aggiornamento e cancellazione dei documenti. Abbiamo distinto bene find() e aggregate(), e abbiamo visto che il primo seleziona documenti mentre il secondo li trasforma per produrre risultati riassuntivi o analitici.

Dal prossimo capitolo inizieremo a lavorare più da vicino sul modello dati e sulle basi operative del dataset aziendale, così da preparare il terreno alle prime query reali.